In [1]:
```
%matplotlib inline
import jax.numpy as jnp
from jax import random
import matplotlib.pyplot as plt
import seaborn as snb

from scipy.stats import wishart
from scipy.stats import multivariate_normal as mvn
from scipy.special import gammaln

from PIL import Image
from exercise10 import Grid2D
from exercise10 import VariationalGMM
from exercise10 import plot_std_dev_contour
from exercise10 import PCA_dim_reduction

# plotting and style stuff
snb.set_style('darkgrid')
colors = snb.color_palette()
snb.set_theme(font_scale=1.1)

# we want to use 64 bit floating precision
import jax
jax.config.update("jax_enable_x64", True)
```

02477 Bayesian Machine Learning - Exercise 10

The purpose of this exercise is to become familiar with variational inference and the coordinate ascent variational inference (CAVI) algorithm in particular. First, we will study CAVI in the context of a simple model, where we will derive all the steps in the algorithm explicitly. Next, we will study Bayesian inference for the Gaussian Mixture model using variational inference and here we will focus more on the application. Finally, you will also become familiar with the Dirichlet and Wishart distributions.

Content

- Part 1: The coordinate ascent variational inference algorithm
- Part 2: Bayesian Gaussian Mixture Model
- Part 3: Variational inference for the Gaussian mixture model
- Part 4: Analyzing a toy data set

Note: The exercise contains several **discussion questions**, which are questions, where are supposed to actively experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs. Instead of proving the full description for every discussion question, we simply tag it with: [**Discussion question**] after the question.

Note: For this exericise, you will need the python module called `PIL`.

Part 1: The coordinate ascent variational inference algorithm

Most models of practical interest leads to intractable posterior distributions, and therefore, we often need to resort to approximations. This week we study **variational inference (VI)** as an alternative to MCMC. The goal of variational inference is to minimize the discrepancy between an approximation $q \in \mathcal{Q}$ and a target distribution of interest p (typically the posterior), where the discrepancy is measured using a **divergence**. In this course, we will focus on the Kullback-Leibler (KL) divergence, such that

$$q^* = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}[q||p],$$

where $q^* \in \mathcal{Q}$ is the **optimal approximation** within a given **variational family** $\mathcal{Q}$ and $\mathrm{KL}[q||p]$ is the KL-divergence between q and p. We optimize over the set of distributions in the variational family and hence, the size of the variational family controls the quality of the approximation. Since probability distributions are functions, we optimize over a function space, which is why this framework is called **variational** inference. A larger/more flexible $\mathcal{Q}$ can yield a better approximation but may be harder to optimize.

It is often convenient to maximize the evidence lower bound (ELBO) instead of minimizing KL. These are equivalent objectives:

$$\begin{aligned} \mathcal{L}(q) &= \mathbb{E}_q[\log p(D, \theta)] - \mathbb{E}_q[\log q(\theta)], \\ \mathrm{KL}[q \,\|\, p(\theta \mid D)] &= \log p(D) - \mathcal{L}(q). \end{aligned}$$

Maximizing $\mathcal{L}(q)$ increases fit to the joint while penalizing complexity in q.

To study variational inference, we will first use it to approximate the posterior distribution of a simple toy model. Consider a dataset given by $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$, where $x_n \in \mathbb{R}$. We assume a Gaussian likelihood for the data, i.e.

$$p(\mathcal{D}|\mu, \tau) = \prod_{n=1}^{N} \mathcal{N}(x_n|\mu, \tau^{-1}),$$

This is equivalent to the likelihood for a linear regression model that only contains an intercept-term, i.e. $\mu \in \mathbb{R}$. Our goal is to estimate the posterior distribution for both the mean $\mu \in \mathbb{R}$ and the precision $\tau > 0$. First, we impose prior distributions over μ and τ:

$$p(\mu, \tau) = p(\mu|\tau)p(\tau) = \mathcal{N}(\mu|\mu_0, (\lambda_0\tau)^{-1})\mathrm{Gamma}(\tau|a_0, b_0),$$

where $\mu_0 \in \mathbb{R}$ and $\lambda_0, a_0, b_0 > 0$ are fixed hyperparameters and

$$\text{Gamma}(\tau|a_0, b_0) = \frac{1}{\Gamma(a_0)} b_0^{a_0} \tau^{a_0-1} \exp(-b_0\tau).$$

So $\mathbb{E}[\tau] = \frac{a_0}{b_0}$ and $\text{Var}[\tau] = \frac{a_0}{b_0^2}$, where $\mathbb{E}[\cdot]$ and $\text{Var}[\cdot]$ denote the expectation and variance, respectively. The prior for μ is a Gaussian distribution with mean μ_0 and precision $\lambda_0\tau$, which is a **conjugate prior** for the Gaussian likelihood. The prior for τ is a Gamma distribution, which is also a conjugate prior for the Gaussian likelihood.

Gaussian priors are common for **location** parameters, and Gamma priors are often used for **precision parameters**.

> **Why this toy model?** This model is actually conjugate (the exact posterior is Normal–Gamma), so we could solve it in closed form. We use it here because it lets us verify VI updates against a known answer and see CAVI in action without algebraic pain.

Mean-field assumption and the CAVI idea

We adopt a mean-field factorization $q(\mu, \tau) = q(\mu)\, q(\tau)$ and maximize the ELBO by alternating updates:

$$\log q^\star(\mu) \;\propto\; \mathbb{E}_{q(\tau)}[\log p(D, \mu, \tau)]\,, \qquad \log q^\star(\tau) \;\propto\; \mathbb{E}_{q(\mu)}[\log p(D, \mu, \tau)]\,.$$

Each update is the conditional of the joint, averaged over the other variable—hence coordinate ascent.

Let

$$\bar{x} = \frac{1}{N}\sum_{n=1}^{N} x_n, \qquad S = \sum_{n=1}^{N} (x_n - \bar{x})^2.$$

Update for $q(\mu)$

Collecting the μ-terms from $\log p(D, \mu, \tau)$ and taking $\mathbb{E}_{q(\tau)}[\cdot]$ yields a Gaussian:

$$q^\star(\mu) = \mathcal{N}\big(\mu \mid m,\ \lambda_\mu^{-1}\big), \qquad \lambda_\mu = (\lambda_0 + N)\, \mathbb{E}_{q(\tau)}[\tau],$$

with mean

$$m = \frac{\lambda_0\mu_0 + N\bar{x}}{\lambda_0 + N}.$$

Interpretation. m is a weighted average of the prior mean μ_0 and the sample mean $\bar{x}$; the posterior precision for μ scales with $\mathbb{E}[\tau]$ and $\lambda_0 + N$.

Update for $q(\tau)$

Collecting the τ-terms and taking $\mathbb{E}_{q(\mu)}[\cdot]$ gives a Gamma:

$$q^\star(\tau) = \text{Gamma}(\tau \mid a, b),$$

with

$$a = a_0 + \frac{N+1}{2}, \qquad b = b_0 + \frac{1}{2}\,\mathbb{E}_{q(\mu)}\left[\sum_{n=1}^{N}(x_n - \mu)^2 + \lambda_0(\mu - \mu_0)^2\right].$$

Using $\sum_n (x_n - \mu)^2 = S + N(\mu - \bar{x})^2$ and $\mathbb{E}_{q(\mu)}[(\mu - c)^2] = \text{Var}_{q(\mu)}(\mu) + \big(m - c\big)^2$,

$$b = b_0 + \frac{1}{2}\left[S + \frac{\lambda_0 N}{\lambda_0 + N}(\bar{x} - \mu_0)^2 + (\lambda_0 + N)\underbrace{\text{Var}_{q(\mu)}(\mu)}_{=\,1/\left((\lambda_0+N)\mathbb{E}_{q(\tau)}[\tau]\right)}\right].$$

Since $\text{Var}_{q(\mu)}(\mu) = 1/\big((\lambda_0 + N)\,\mathbb{E}_{q(\tau)}[\tau]\big)$, the last term simplifies to $1/\mathbb{E}_{q(\tau)}[\tau]$. This is the only place the two updates "talk," so we iterate.

Moments used each step:

$$\mathbb{E}_{q(\tau)}[\tau] = \frac{a}{b} \quad \text{(Gamma shape–rate)}, \qquad \text{Var}_{q(\mu)}(\mu) = \frac{1}{(\lambda_0 + N)\,\mathbb{E}_{q(\tau)}[\tau]}.$$

The CAVI loop (for this model)

1. **Initialize** $\mathbb{E}_{q(\tau)}[\tau]$ (e.g., $\frac{a_0}{b_0}$).

2. **Update** $q(\mu)$**:**

$$m \leftarrow \frac{\lambda_0 \mu_0 + N\bar{x}}{\lambda_0 + N}, \qquad \lambda_\mu \leftarrow (\lambda_0 + N)\,\mathbb{E}_{q(\tau)}[\tau].$$

3. **Update $q(\tau)$:**

$$a \leftarrow a_0 + \frac{N+1}{2}, \qquad b \leftarrow b_0 + \frac{1}{2}\left[S + \frac{\lambda_0 N}{\lambda_0 + N}(\bar{x} - \mu_0)^2 + \frac{1}{\mathbb{E}_{q(\tau)}[\tau]}\right],$$

   then set $\mathbb{E}_{q(\tau)}[\tau] \leftarrow \frac{a}{b}$.

4. **Repeat** steps 2–3 until convergence of the ELBO or the moments.

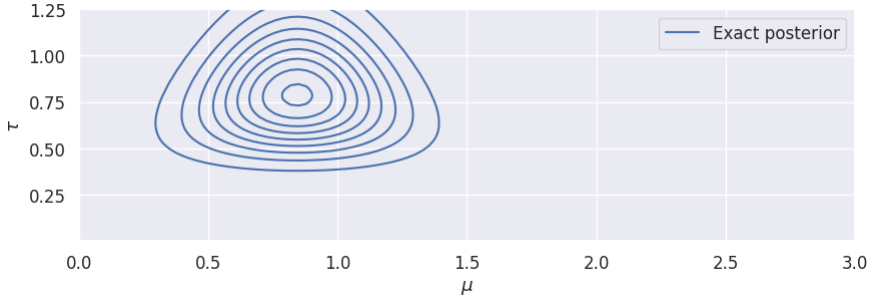
Checks & intuition.

- The mean m doesn't change across iterations—it equals the conjugate posterior mean.
- As $\mathbb{E}[\tau]$ grows (higher precision), $q(\mu)$ contracts.
- Because this model is conjugate, the CAVI fixed point matches the exact Normal–Gamma posterior; for nonconjugate models, CAVI still applies but updates typically involve intractable expectations you approximate.

Parameterization reminder. We use **shape–rate** for the Gamma: $\mathrm{Gamma}(a, b)$ has mean a/b. If you prefer **shape–scale** (k, θ), convert via $b = 1/\theta$.

Let's generate a synthetic dataset and visualize the exact posterior, $p(\mu, \tau|\mathcal{D})$. To do that, we will re-use the `Grid2D` -class from week 2.

In [12]:
```
# implement log PDFs for Gaussian and gamma distributions
# log PDF of univariate normal distribution:
# log N(x | m, v) = -0.5 * log(2πv) - 0.5 * (x - m)^2 / v
log_npdf = lambda x, m, v: -0.5*jnp.log(2*jnp.pi*v) - 0.5*(x-m)**2/v

# log PDF of gamma distribution:
# log Gamma(x | a0, b0) = -gammaln(a0) + a0*log(b0) + (a0-1)*log(x) - b0*x
log_gamma = lambda x, a0, b0: -gammaln(a0) + a0*jnp.log(b0) + (a0-1)*jnp.log(x) - b0 *x

# set seed
key = random.PRNGKey(1)

# create synthetic data set
N = 20
x = 1 + jnp.sqrt(0.5*jnp.pi)*random.normal(key, shape=(N,))

# specify hyperparameters of the model
lambda0 = 1.
mu0 = -1.
a0 = 1.
b0 = 1.

# implement exact posterior up to a constant (i.e. the joint distribution)
def exact_posterior(mu, tau):
    """ evaluate and  returns the exact posterior distribution p(mu, tau|D) up to a constant """
    log_lik = log_npdf(x, mu, 1/tau)
    log_prior = log_gamma(tau, a0, b0) + log_npdf(mu, mu0, 1./(lambda0*tau))
    return log_lik.sum(axis=-1)  + log_prior.sum(axis=-1)

# define grid for plotting
mus = jnp.linspace(0, 3, 100)
taus = jnp.linspace(1e-7, 1.25, 101)
exact_posterior_grid = Grid2D(mus, taus, exact_posterior, name='Exact posterior')

# visualize
fig, ax = plt.subplots(1, 1, figsize=(10, 3))
exact_posterior_grid.plot_contours(ax, f=jnp.exp)
ax.legend();
```



This simple model is a **conjugate model** and hence, we can derive the exact posterior distribution analytically. However, to become familiar with variational inference, we will approximate the posterior distribution $p(\mu, \tau|\mathcal{D}) \approx q(\mu, \tau)$ using a variational approximation $q(\mu, \tau)$. Specially, we will assume a **factorized** variational family satisfying:

$$q(\mu, \tau) = q(\mu)q(\tau).$$

That is, we choose $\mathcal{Q}$ to be the set of all distributions $q(\mu, \tau)$, where μ and τ are **independent**. However, we do **not** assume anything about the shapes or functional forms of $q(\mu)$ or $q(\tau)$.

We will use the **coordinate ascent variational inference** (CAVI) algorithm to compute the optimal factors $q(\mu)$ and $q(\tau)$, which states that:

$$\ln q^*(\mu) = \mathbb{E}_{q(\tau)}\left[\ln p(\mathcal{D}, \mu, \tau)\right] + \text{constant}$$
$$\ln q^*(\tau) = \mathbb{E}_{q(\mu)}\left[\ln p(\mathcal{D}, \mu, \tau)\right] + \text{constant},$$

where $p(\mathcal{D}, \mu, \tau)$ is the joint distribution for the model. "constant" means any term not depending on the variable being updated (it can be dropped when identifying the distributional form). These updates are equivalent to maximizing the ELBO and simply say: hold one factor fixed, take the expected log-joint over it, and exponentiate. Before computing the expectations above, we will prepare the expression for the joint distribution, which is the central object when working with variational inference.

Prepare the joint (the thing we keep reusing)

Write the joint as

$$p(D, \mu, \tau) = p(D \mid \mu, \tau)\, p(\mu \mid \tau)\, p(\tau).$$

Using our model and the **shape–rate** Gamma parameterization

$$\text{Gamma}(\tau \mid a_0, b_0) = \frac{b_0^{a_0}}{\Gamma(a_0)}\, \tau^{a_0 - 1} e^{-b_0 \tau},$$

we have

$$\log p(D, \mu, \tau) = \underbrace{\sum_{n=1}^{N} \log \mathcal{N}(x_n \mid \mu, \tau^{-1})}_{\text{likelihood}} + \underbrace{\log \mathcal{N}(\mu \mid \mu_0, (\lambda_0 \tau)^{-1})}_{\text{prior on } \mu} + \underbrace{\log \text{Gamma}(\tau \mid a_0, b_0)}_{\text{prior on } \tau}.$$

Convenient data summaries:

$$\bar{x} = \frac{1}{N}\sum_{n=1}^{N} x_n, \qquad S = \sum_{n=1}^{N} (x_n - \bar{x})^2, \qquad \sum_{n=1}^{N} (x_n - \mu)^2 = S + N(\mu - \bar{x})^2.$$

Expanding the log-joint and discarding terms constant in both μ and τ,

$$\boxed{\log p(D, \mu, \tau) = \left(\tfrac{N}{2} + a_0 - 1\right) \log \tau - \frac{\tau}{2}\left[S + N(\mu - \bar{x})^2 + \lambda_0(\mu - \mu_0)^2\right] - b_0 \tau + \text{const.}}$$

This form makes the two updates transparent:

- As a function of μ (holding τ inside an expectation), the log-joint is quadratic in $\mu \Rightarrow q^*(\mu)$ is Gaussian.
- As a function of τ (holding μ inside an expectation), it is $(\cdot) \log \tau - \tau(\cdot) \Rightarrow q^*(\tau)$ is Gamma.

Plugging the joint into the CAVI updates

Update for $q(\mu)$: take $\mathbb{E}_{q(\tau)}[\cdot]$ of the boxed expression and keep only μ-terms. Completing the square yields

$$q^*(\mu) = \mathcal{N}\left(m,\ \frac{1}{(\lambda_0 + N)\,\mathbb{E}_{q(\tau)}[\tau]}\right), \qquad m = \frac{\lambda_0\mu_0 + N\bar{x}}{\lambda_0 + N}.$$

Reading guide: m is a weighted average of prior mean and sample mean; higher $\mathbb{E}[\tau]$ (more precision) tightens $q(\mu)$.

Update for $q(\tau)$: take $\mathbb{E}_{q(\mu)}[\cdot]$ and keep only τ-terms. Using $\mathbb{E}_{q(\mu)}[(\mu - c)^2] = \mathrm{Var}_{q(\mu)}(\mu) + (m - c)^2$,

$$q^*(\tau) = \mathrm{Gamma}(\tau \mid a, b), \quad a = a_0 + \frac{N+1}{2}, \quad b = b_0 + \frac{1}{2}\,\mathbb{E}_{q(\mu)}\big[S + N(\mu - \bar{x})^2 + \lambda_0(\mu - \mu_0)^2\big].$$

Substituting $\mathrm{Var}_{q(\mu)}(\mu) = 1/[(\lambda_0 + N)\,\mathbb{E}_{q(\tau)}[\tau]]$ and $\sum(x_n - \mu)^2 = S + N(\mu - \bar{x})^2$ gives a closed form for b in $\bar{x}, S, \mu_0, \lambda_0, a_0, b_0$.

What "const." hides (and why it's safe): "Const." means "does not depend on the variable being updated," so it vanishes upon exponentiating and normalizing to identify the distribution.

Task 1.1: **(Optional exercise!)** Show that the logarithm of the joint distribution is given by

$$\ln p(\mathcal{D}, \mu, \tau) = \left[a_0 + \frac{N+1}{2} - 1\right]\ln\tau - \left[b_0 + \frac{N\bar{x^2}}{2} - \mu(N\bar{x} + \lambda_0\mu_0) + \frac{1}{2}\mu^2(N + \lambda_0) + \frac{\lambda_0}{2}\mu_0^2\right]\tau + K,$$

where K is an additive constant independent of μ, τ and we have defined $N\bar{x} = \sum_{n=1}^{N} x_i$, and $N\bar{x^2} = \sum_{n=1}^{N} x_i^2$ to simplify the expressions.

Hints:

- *This is an **optional** exercise designed to practice the process of deriving and simplifying the log joint distribution. However, due to the amount of algebraic manipulations needed, it can be time consuming. Therefore, you can consider skipping it initially and then come back to it if you finish early.*
- *If you decide to skip the task completely, then at least think about how you would solve it and consider checking your understanding with one of the TAs.*

Task 1.2: Show that $\ln q^*(\mu) = \mu(N\bar{x} + \lambda_0\mu_0)\mathbb{E}_{q(\tau)}[\tau] - \frac{1}{2}\mu^2(N + \lambda_0)\mathbb{E}_{q(\tau)}[\tau] + \text{const}$

*Hints: Start with the definition for $\ln q^(\mu). $Insert the expression for the log joint distribution and absorb all terms independent of μ into the additive constant.**

Task 1.3: Argue that $q^*(\mu)$ **must** be a Gaussian distribution and identify its mean m and variance v such that $q^*(\mu) = \mathcal{N}(\mu|m, v)$.

Task 1.4: Show that $\ln q^*(\tau)$ has the functional form of a Gamma distribution, i.e. $q^*(\tau) = \mathrm{Gamma}(\tau|a, b)$, and identify the parameters a, b.

Hint: The functional form of a Gamma-distribution is $\ln \mathrm{Gamma}(\tau|a, b) = (a - 1)\log(\tau) - b\tau + K$.

We have now shown that

$$q^*(\mu) = \mathcal{N}(\mu|m, v)$$
$$q^*(\tau) = \mathrm{Gamma}(\tau|a, b),$$

where we refer to m, v, a, b as **variational parameters**.

We can now compute the optimal parameters for m, v, a, b using the following steps:

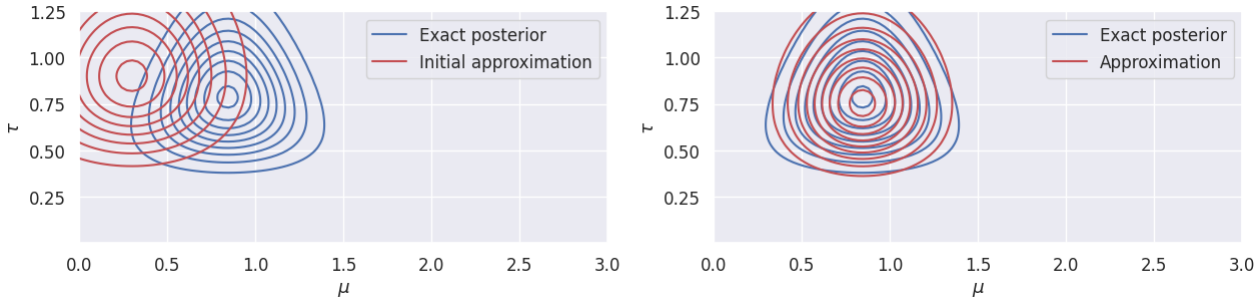
- Initialize $q(\mu)$ and $q(\tau)$ by initializing m, v, a, b
- Do until convergence or maximum number of iterations
  - Update $q^*(\mu)$ by computing m, v
  - Update $q^*(\tau)$ by computing a, b

Task 1.5: Complete the implementation of the function `fit_approx` in the class `VariationalApproximation` below.

Hints: For $\tau \sim \mathrm{Gamma}(\tau|a, b)$, the expected value is $\mathbb{E}[\tau] = \frac{a}{b}$.

In [13]:
```python
class VariationalApproximation(object):

    def __init__(self, x, mu0=0, lambda0=0, a0=1, b0=1):
        # store data & hyperparameters
        self.x = x
        self.N = len(x)
        self.mu0 = mu0
        self.lambda0 = lambda0
        self.a0 = a0
        self.b0 = b0

    def initialize(self, m, v, a, b):
        self.m = m
        self.v = v
        self.a = a
        self.b = b
```

```
        return self

    def fit_approx(self, num_iterations):
        """ Computes the optimal values for variational parameters (m, v, a,b) using CAVI by iteratively updating the components q(mu) = N(mu|m, v) and q(tau) = Gamma(tau|a,b) """

        # precompute
        xbar = jnp.mean(self.x)
        x2bar = jnp.mean(self.x**2)

        # initialize parameters
        m, v = self.m, self.v
        a, b = self.a, self.b

        for itt in range(num_iterations):

            ############################################
            # Your solution goes here
            ############################################

            # update q(mu)
            tau_mean = a / b
            m = (N * xbar + self.lambda0 * self.mu0) / (self.N + self.lambda0)
            v = 1/ (tau_mean*(self.N + self.lambda0))

            # update q(tau)
            mu_mean = m
            mu2_mean = m**2 + v
            a = self.a0 + (self.N+1)/2
            b = self.b0 + 0.5*self.N*x2bar - mu_mean*(self.N*xbar + self.lambda0*self.mu0) + 0.5*mu2_mean*(self.N + self.lambda0) + 0.5*self.lambda0*self.mu0**2

            ############################################
            # End of solution
            ############################################

        # sanity check and store optimal variational parameters
        assert v > 0, f"The parameter v must be positive scalar, but the actual value for v was {v:4.3f}. Check your implementation."
        assert a > 0, f"The parameter a must be positive scalar, but the actual value for a was {a:4.3f}. Check your implementation."
        assert b > 0, f"The parameter b must be positive scalar, but the actual value for b was {b:4.3f}. Check your implementation."
        self.a, self.b = a, b
        self.m, self.v = m, v
        return self


    def pdf(self, mu, tau):
        return log_npdf(mu, self.m, self.v).sum(axis=2) + log_gamma(tau, self.a, self.b).sum(axis=2)

# sanity check of results
approx = VariationalApproximation(jnp.array([-1, 1]), mu0, lambda0, a0, b0).initialize(0, 1, 1, 1).fit_approx(100)
assert jnp.allclose(jnp.array([approx.m, approx.v, approx.a, approx.b]), jnp.array([-0.33333333,  0.38888889,  2.5       ,  2.91666667])), 'Expected m, v, a, b. Please check your implementation.'

# initialize approximation
approx = VariationalApproximation(x, mu0, lambda0, a0, b0)
approx.initialize(m=0.3, v=0.1, a=10, b=10)
init_approx_grid = Grid2D(mus, taus, approx.pdf, name='Initial approximation')

# optimize approximation
approx.fit_approx(num_iterations=100)
approx_grid = Grid2D(mus, taus, approx.pdf, name='Approximation')

# visualize
fig, ax = plt.subplots(1, 2, figsize=(15, 3))
exact_posterior_grid.plot_contours(ax[0], f=jnp.exp)
exact_posterior_grid.plot_contours(ax[1], f=jnp.exp)
init_approx_grid.plot_contours(ax[0], f=jnp.exp, color='r')
approx_grid.plot_contours(ax[1], f=jnp.exp, color='r')
ax[0].legend()
ax[1].legend();
```



If everything was implemented correctly, you should see a plot of the approximate posterior before (left panel) and after (right panel) the optimization process.

Task 1.6: What is the approximate marginal distribution of $q(\mu)$ and $q(\tau)$?

Solution

For factorized and mean-field variational families, marginals are straight-forward:

$$q(\mu) = \int q(\mu, \tau) \mathrm{d}\tau = \int q(\mu) q(\tau) \mathrm{d}\tau = q(\mu) \int q(\tau) \mathrm{d}\tau = q(\mu) = \mathcal{N}(\mu | m, v)$$
$$q(\tau) = \int q(\mu, \tau) \mathrm{d}\mu = \int q(\mu) q(\tau) \mathrm{d}\mu = q(\tau) \int q(\mu) \mathrm{d}\mu = q(\tau) = \mathrm{Gamma}(\tau | a, b)$$

End of solution

Part 2: Bayesian Gaussian Mixture Model

The Gaussian mixture model is very common in machine learning and statistics with a multitude of applications in clustering, density estimation, and outlier detection. It is also often used a building blocks in more complicated models.

The Gaussian mixture model (GMM) for D-dimensional data is a weighted mixture of K Gaussians:

$$p(\mathbf{x}_n | \pi, \mathbf{m}, \Lambda) = \sum_{k=1}^{K} \pi_k N(\mathbf{x}_n | \mathbf{m}_k, \Lambda_k^{-1}), \tag{1}$$

where $\mathbf{m}_k \in \mathbb{D}^D$ and $\Lambda_k \in \mathbb{R}^{D \times D}$ are the mean vector and precision matrix for the k'th component. The parameters $\pi_k \in [0, 1]$ for $k = 1, \ldots, K$ are the **mixing weights** and satisfy $\sum_{k=1}^{K} \pi_k = 1$. Intuitively, π_k represents the fraction of samples belonging to the k'th component.

The GMM as a latent variable model

The GMM can also be formulated as a latent variable model with latent variables z_n such that

$$p(\mathbf{x}_n | z_n, m, \Lambda) = \prod_{k=1}^{K} N(\mathbf{x}_n | \mathbf{m}_k, \Lambda_k^{-1})^{z_{nk}} \tag{2}$$

where z_n is a K-dimensional one-hot encoded binary vector indicating the component index for the n'th observation. This implies

$$p(\mathbf{x}_n | z_n = j, m, \Lambda) = N(\mathbf{x}_n | m_j, \Lambda_j^{-1})$$

Note that we often abuse the notation and write e.g. $z_n = 3$ as a shorthand notation for $z_n = [0 \quad 0 \quad 1 \quad 0 \ldots 0]$ and so on. The distribution of the latent variables is given by

$$p(z_n | \pi) = \mathrm{Categorial}(z_n | \pi) = \prod_{k=1}^{K} \pi_k^{z_{nk}}, \tag{3}$$

where π_k is the proportion of sample from the k'th component, i.e. $p(z_n = j | \pi) = \pi_j$. The variable z_n is called a **latent variable** because we can't observe it, we can only observe $\mathbf{x}_n$.

We can derive the marginalized formulation in eq. (1) from the latent variable formulation in eq. (2)-(3) by applying to sum rule to marginalize over the latent variable z_n, i.e. $p(\mathbf{x}_n) = \sum_k p(\mathbf{x}_n | z_n = k) p(z_n = k)$.

Maximum likelihood

The classic way to estimate the model parameters for GMMs (m_k, Λ_k, π_k for $k = 1, \ldots, K$) is through maximum likelihood estimation via the Expectation-Maximimization (EM) algorithm (not part of the curriculum of this course). However, the maximum likelihood solution is not well-posed and can suffer from singularities when a component "collapses" on a single data point. This causes the variance component for the collapsing component to go to zero and eventually the algorithm will crash.

Imposing priors

We can avoid these issues by using Bayesian inference to estimate the parameters. As usual, we impose prior distributions on the parameters: m_k, Λ_k, and π_k:

$$\begin{aligned}
\pi &\sim \text{Dirichlet}(\alpha_0) \\
\Lambda_k &\sim \text{Wishart}(W_0, \nu_0) \\
\mu_k|\Lambda_k &\sim \text{Normal}(m_0, (\beta_0\Lambda_k)^{-1}) \\
z_n|\pi &\sim \text{Categorical}(\pi) \\
x_n|\mu, \Lambda, z_n &\sim \text{Normal}(\mu_{z_n}, \Lambda_{z_n}^{-1}),
\end{aligned}$$

where we consider all parameters with subscript '0' as hyperparameters.

Task 2.1: What is the sample space for each of the distributions above?

Generative story

We can interret and visualize the assumptions of the model by considering the generative story of the model. Assume the number of observations N and the number of components K are fixed. By using Bayesian GMM to model a data set, we assume that the data set has been generated in the following way:

1. Nature samples the mixing weights $\pi \sim \text{Dirichlet}(\alpha_0)$
2. Nature samples a precision matrix $\Lambda_k \sim \text{Wishart}(W_0, \nu_0)$ for each component $k = 1, \ldots, K$
3. Nature then samples a mean $m_k|\Lambda_k \sim \text{Normal}(m_0, (\beta_0\Lambda_k)^{-1})$ for each component $k = 1, \ldots, K$ conditioned on Λ_k
4. For each data point, nature samples a component index $z_n|\pi \sim \text{Categorical}(\pi)$
5. For each data point, nature samples an observation from the relevant cluster distribution $x_n|\mu, \Lambda, z_n \sim \text{Normal}(\mu_{z_n}, \Lambda_{z_n}^{-1})$

We can use **ancestral sampling** to generate synthetic data sets from this model

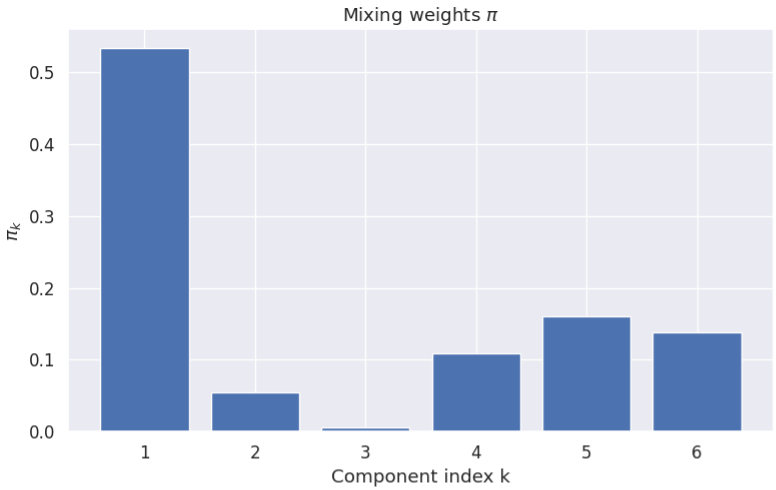
Task 2.2: Complete the implementation of ancestral sampling below.

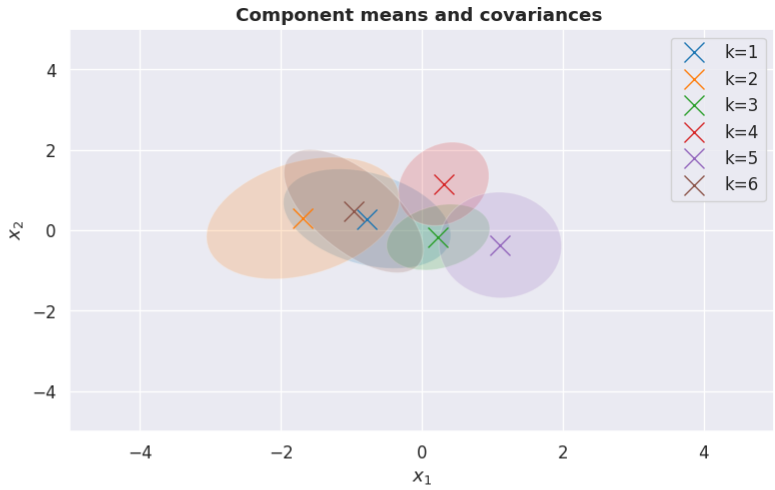
Hints:

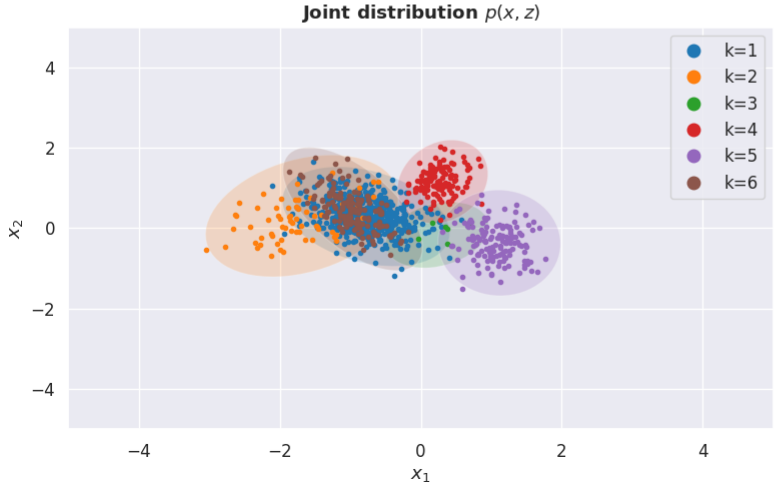
- *The functions `random.dirichlet`, `random.choice`, and `scipy.stats.wishart.rvs` will be handy. Note that the Wishart distribution is not supported by JaX, and therefore, we will rely on the `scipy.stats` module instead.*
- *Recall: everytime we want to generate a random number in JaX, we need to explicitly provide a `key` to specify the state of the random number generator. The code below prepares a unique key for each sampling step. That is, `key_pi` is intended to be used when sampling π, `key_m` is an array of keys intended to be used when sampling each of the m_k and so on and so forth. See https://docs.jax.dev/en/latest/random-numbers.html for more info.*

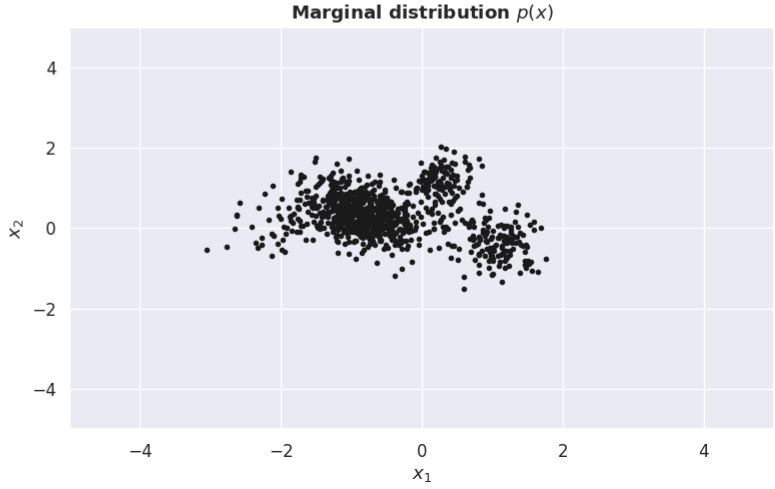
In [14]:
```
# number of components K,  dimension D, and number of observations N
K = 6
D = 2
N = 1000

# prepare keys for random number generations
key = random.PRNGKey(123)
key_pi, key_m, key_z, key_x = random.split(key, num=4)
key_m = random.split(key_m, num=K) # split into array of K keys, one for each m_k
key_x = random.split(key_x, num=N) # split into array of N keys, one for each x_n

# hyperparameter for the Dirichlet distribution
alpha0 = 0.5*jnp.ones(K)

# hyperparameters for the components
m0 = jnp.zeros(D)
W0 = jnp.identity(D)
nu0 = 10
beta0 = 0.1

# Step 1: sample mixing weights (jnp array with shape (K,))
pi = random.dirichlet(key_pi, alpha0)

# Step 2: sample prior precision matrix for each component (jnp array with shape (K, D, D))
Lambda_k = wishart.rvs(nu0, W0, size=K)

# Step 3: sample prior mean vector for each component
Sk = []
mk = []
for k in range(K):
    Sk.append(jnp.linalg.inv(Lambda_k[k]))
    mk.append(random.multivariate_normal(key_m[k], m0, Sk[k]/beta0))

# Step 4: sample latent variable for each observation (jnp array with shape (N, ))
z = random.choice(key_z, jnp.arange(K), shape=(N, ), p=pi)

# Step 5: sample observations for all n (jnp array with shape (N, D))
X = []
for n in range(N):
```

```
    X.append(random.multivariate_normal(key_x[n], mk[z[n]], Sk[z[n]]))
X = jnp.stack(X)

# Visualize
fig, axes = plt.subplots(2, 2, figsize=(20, 12))

# plot mixing weights
axes[0,0].bar(jnp.arange(1,K+1), pi)
axes[0,0].set(xlabel='Component index k', ylabel = '$\\pi_k$', xticks=jnp.arange(1, K+1))
axes[0,0].set_title('Mixing weights $\\pi$')

# plot component mean and covariances
for k in range(K):
    axes[0,1].plot(mk[k][0], mk[k][1], 'x', color=colors[k], markersize=15, label='k=%d' % (k+1))
    plot_std_dev_contour(axes[0, 1], mk[k], Sk[k], facecolor=colors[k], alpha=0.2)
    plot_std_dev_contour(axes[1, 0], mk[k], Sk[k], facecolor=colors[k], alpha=0.2)
    axes[1,0].plot(X[z==k, 0], X[z==k, 1], '.', color=colors[k], label='k=%d' % (k+1))

axes[0,1].set_title('Component means and covariances ', fontweight='bold')
axes[0,1].legend()
axes[1,0].set_title('Joint distribution $p(x, z)$', fontweight='bold');
axes[1,0].legend(markerscale=3)

# plot observations
axes[1,1].plot(X[:, 0], X[:, 1], '.', color='k')
axes[1,1].set_title('Marginal distribution $p(x)$', fontweight='bold');

# add labels etc
for a in [1, 2, 3]:
    axes.flat[a].set(xlim = (-5, 5), ylim=(-5, 5), xlabel='$x_1$', ylabel='$x_2$')

fig.subplots_adjust(hspace=0.3)
```









Task 2.3: Why is the Dirichlet distribution sometimes called a "distribution over distributions"? [**Discussion question**]

Task 2.4: How does the hyperparameter α_0 affect the generated samples? Explore the generated datasets for different values of α_0, e.g. $\alpha_0 = 0.2 \cdot \mathbf{1}$, $\alpha_0 = 1 \cdot \mathbf{1}$, and $\alpha_0 = 10 \cdot \mathbf{1}$, where $\mathbf{1}$ is a K-dimensional vector of ones. Inspect the results for a few different random seeds. [**Discussion question**]

Task 2.5: Relate the panels in the figure above to the "generative story" explained above. [**Discussion question**]

Task 2.6: Which hyperparameter do we need to change in order to increase the separation between clusters, i.e. to increase the distance between the means of each cluster? [**Discussion question**]

Part 3: Variational inference for the Gaussian mixture model

We are now ready to set-up the variational approximation for the mixture model. The joint distribution of observed data $\mathbf{X}$, latent variable $\mathbf{Z}$ and model parameters μ, Λ, π follows from the **product rule**:

$$p(X, Z, \mu, \Lambda, \pi) = \prod_{n=1}^{N} \prod_{k=1}^{K} \mathcal{N}(x_n | \mu_k, \Lambda_k^{-1})^{z_{nk}} \prod_{n=1}^{N} \mathrm{Cat}(z_n | \pi) \, \mathrm{Dir}(\pi | \alpha) \prod_{k=1}^{K} p(\mu_k, \Lambda_k),$$

where

$$p(\mu_k, \Lambda_k) = p(\mu_k|\Lambda_k)p(\Lambda_k) = \mathcal{N}(\mu_k|m_0, [\beta_0\Lambda_k]^{-1})\mathcal{W}(\Lambda_k|W_0, \nu_0).$$

Our goal is to compute the posterior distribution of the parameters conditioned on the data X, i.e. $p(Z, \mu, \Lambda, \pi|X)$. However, computing the exact posterior distribution for this model would require us to sum over all K^N possible configurations of the latent variables z_n, which becomes infeasible for even small to moderate sized datasets. Therefore, we have to resort to approximate inference. In this exercise, we will work with **variational approximations**, but one could also implement an MH-sampler or derive a Gibbs sampler for the model.

Informally, we define a collection of tractable distributions $\mathcal{Q}$ and search for the distribution $q \in \mathcal{Q}$ that resembles the true posterior distribution as close as possible as measured by the **Kullback-Leibler (KL) divergence** $\mathrm{KL}[q||p]$.

For the Baysian GMM, we choose the variational family $\mathcal{Q}$ to be the collection of all distributions with following factorization

$$q(Z, \mu, \Lambda, \pi) = q(Z)q(\mu, \Lambda, \pi)$$

That is, we assume that the latent variables z_n are independent from the rest of variables and then we search for the best matching distribution under this independence assumption. Note that this is the only assumption we need in order to be able to derive a variational approximation for the posterior.

To identify the best matching distribution $q^* \in \mathcal{Q}$, we minimize the KL-divergence between $q \in \mathcal{Q}$ and $p \equiv p(Z, \mu, \Lambda, \pi|X)$

$$q^* = \arg\min_{q\in\mathcal{Q}} \mathrm{KL}[q||p]. \tag{4}$$

As shown in the lecture, the KL-divergence $\mathrm{KL}[q||p]$ can we re-written as

$$\mathrm{KL}[q||p] = \ln p(X) - \mathcal{L}[q] \qquad \Longleftrightarrow \qquad \ln p(X) = \mathrm{KL}[q||p] + \mathcal{L}[q], \tag{5}$$

where $p(X)$ is the marginal likelihood and the $\mathcal{L}[q]$ is the **evidence lowerbound (ELBO)** defined as

$$\mathcal{L}[q] = \mathbb{E}_q\left[\ln p(X, Z, \mu, \Lambda, \pi)\right] - \mathbb{E}_q\left[\ln q(Z, \mu, \Lambda, \pi)\right] \tag{6}$$

In practice, we optimize the **lower bound** $\mathcal{L}[q]$ rather than the KL divergence

$$q^* = \arg\max_{q\in\mathcal{Q}} \mathcal{L}[q]. \tag{7}$$

Using the CAVI algorithm leads to iterating the following two steps:

$$\ln q(Z) \propto \mathbb{E}_{q(\mu,\Lambda,\pi)}\left[\ln p(X, Z, \mu, \Lambda, \pi)\right] \tag{8}$$
$$\ln q(\pi, \mu, \Lambda) \propto \mathbb{E}_{q(Z)}\left[\ln p(X, Z, \mu, \Lambda, \pi)\right]. \tag{9}$$

We fit the approximation by optimizing the lower bound $\mathcal{L}[q]$. Therefore, we can evaluate and monitor $\mathcal{L}[q]$ using eq. (6) to check when the optimization has converged.

Eq. (8) and (9) above can be reduced to a set of simple update equations as done in Bishop from eq. (10.43) to (10.68). In this course, we won't dive into these specific calculations, instead we will focus on understanding the general methodology. That is, you should be able to understand and explain the ideas behind equations eq. (4)-(9) from a conceptual point of view, but the steps from eq. (8)-(9) to eq. (10) will not be part of the curriculum.

Don't worry if these concepts appear very abstract now, we will see more examples in the following two weeks, but make sure the discuss the key equations with one of the teachers, if you find them confusing.

The optimal variational approximation takes the form

$$q(Z, \pi, \mu, \Lambda) = q(Z)q(\pi)q(\mu, \Lambda) = \underbrace{\prod_{n=1}^{N} \mathrm{Categorical}(z_n|r_n)}_{q(Z)} \underbrace{\mathrm{Dir}(\pi|\alpha)}_{q(\pi)} \underbrace{\prod_{k=1}^{K} \mathcal{N}\left(\mu_k|m_k, [\beta_k\Lambda_k]^{-1}\right)\mathcal{W}(\Lambda_k|W_k, \nu_k)}_{q(\mu,\Lambda)} \tag{10}$$

The variable Z represents the cluster assignments, and the (approximate) posterior distribution $Q(Z)$ represents the distribution over cluster assignments for each data point after having seen the data. The posterior cluster assignment for the n'th datapoint is $q(z_n) = \mathrm{Categorical}(z_n|r_n)$, where z_n is cluster index (and sometimes represented as a one-hot encoded vector) and r_n is a vector probabilities such that probability of the n'th data point being in cluster k is $r_{n,k}$. These probabilities are often called ***responsibilities*** as the number $r_{n,k}$ can be interpreted as how much the k'th cluster is responsible for explaning the n'th data point.

We often defined the **effective number of data points** in the k'th cluster to be

$$N_k = \sum_{n=1}^{N} r_{n,k} \qquad \text{(eq. (10.51) in Bishop)}$$

The posterior distribution of the mixing weights is $Q(\pi) = \mathrm{Dir}(\pi|\alpha)$, where $\alpha \in \mathbb{R}_+^D$ is as vector of parameters for the Dirichlet distribution, where $a_k = a_0 + N_k$. The posterior mean of the k'th mixing weight is therefore

$$\mathbb{E}\left[\pi_k|X\right] = \frac{\alpha_0 + N_k}{K\alpha_0 + N}. \qquad \text{(eq. (10.69) in Bishop)}$$

Note that there is a typo in the corresponding equation in Bishop eq. (10.69). The posterior distribution for the k'th cluster mean and cluster precision is $q(\mu_k|\Lambda_k) = \mathcal{N}(\mu_k|m_k, [\beta_k\Lambda_k]^{-1})$ and $q(\Lambda_k) = \mathcal{W}(W_k, \nu_k)$, respectively, where the parameters are given by

$$
\begin{aligned}
\beta_k &= \beta_0 + N_k \\
m_k &= \frac{1}{\beta_k}(\beta_0 m_0 + N_k \bar{x}_k) \\
W_k^{-1} &= W_0^{-1} + N_k S_k + \frac{\beta_0 N_k}{\beta_0 + N_k}(\bar{x}_k - m_0)(\bar{x}_k - m_0)^T \\
\nu_k &= N_k + 1
\end{aligned} \tag{10b}
$$

for

$$
\begin{aligned}
\bar{x}_k &= \frac{1}{N_k}\sum_{n=1}^{N} r_{n,k} x_n \\
S_k &= \frac{1}{N_k}\sum_{n=1}^{N} r_{n,k}(x_n - \bar{x}_n)(x_n - \bar{x}_n)^T
\end{aligned}
$$

Task 3.1: Use eq. (5) to explain why $\mathcal{L}[q]$ is a lower bound on $\ln p(X)$. [**Discussion question**]

Task 3.2: Use eq. (5) to explain why maximizing the lower bound $\mathcal{L}[q]$ is equivalent to minimizing the KL-divergence wrt. q [**Discussion question**]

Task 3.3: Show that the posterior mean m_k for each cluster is a convex combination of the prior mean and the weighed average of the data points $\bar{x}_k$. That is, show that there exist some value $\rho \in [0, 1]$ such that

$$
m_k = (1 - \rho)m_0 + \rho \bar{x}_k
$$

Hints:

- Argue that $\rho = \frac{N_k}{\beta_k} \in [0, 1]$ and $\frac{\beta_0}{\beta_k} = 1 - \rho$.
- If you don't know how to get started here, do not hesitate to ask for help :-)

Task 3.4: **Optional** Show that the posterior mean for cluster k converges to the prior mean when $\beta_0 \to \infty$ using eq. (10.b)

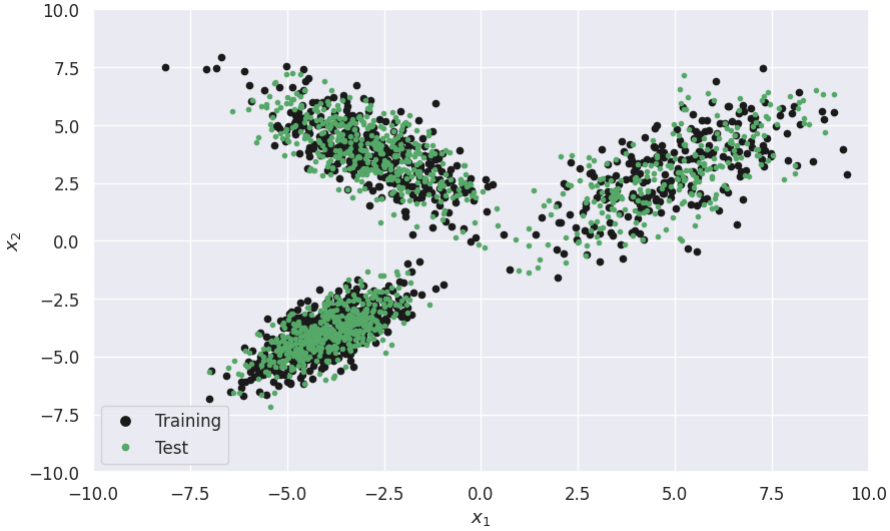
Hints

- What happens to β_k when $\beta_0 \to \infty$?
- If you don't know how to get started here, do not hesitate to ask for help :-)

Part 4: Analyzing a toy data set

The class `VariationalGMM` in the module `exercise10.py` provides a basic implementation of variational approximation of the GMM described above. First, we will use the Variational GMM algorithm to analyze a simple toy dataset:

In [15]:
```
# load data
data = jnp.load('./exercise10_toydata.npz')
X = data['X']
N = len(X)

# split data in training/test
key = random.PRNGKey(123)
Ntrain = int(0.5*N)
Ntest = N - Ntrain
test_idx = random.choice(key, jnp.arange(N), replace=False, shape=(Ntest, ))
train_idx = jnp.setdiff1d(jnp.arange(N), test_idx)
Xtrain, Xtest = X[train_idx], X[test_idx]

# plot data
def plot_data(ax=None, train=True, test=True, legend=True):
    if ax is None:
        fig, ax = plt.subplots(1, 1, figsize=(10, 6))
    if train:
        ax.plot(Xtrain[:, 0], Xtrain[:, 1], 'k.', label='Training', markersize=9)
    if test:
        ax.plot(Xtest[:, 0], Xtest[:, 1], 'g.', label='Test')
    if legend:
        ax.legend(markerscale=1.5)
    ax.set(xlabel='$x_1$', ylabel='$x_2$', xlim=(-10, 10), ylim=(-10, 10))

plot_data()
```

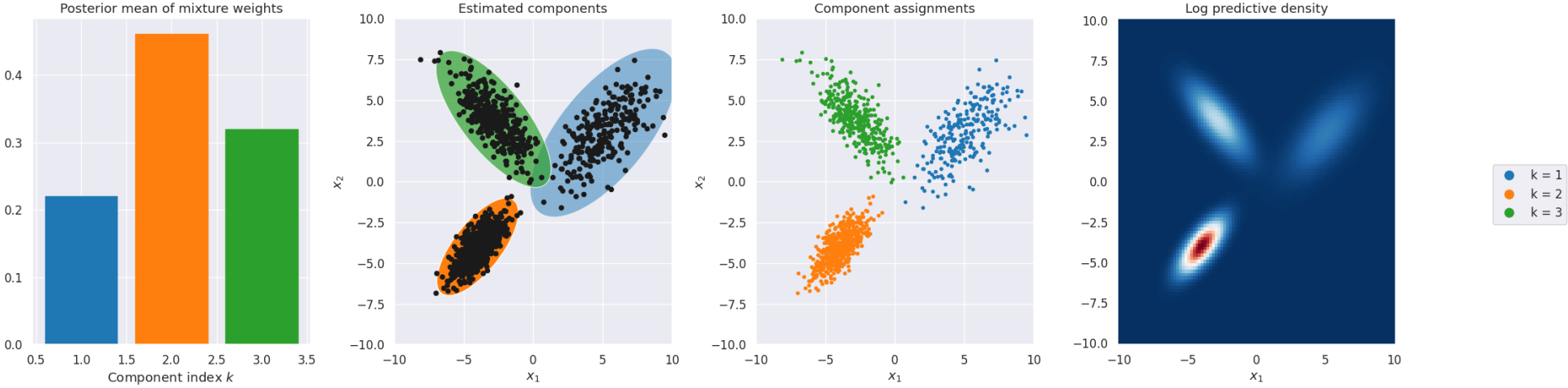


By visual inspection of the data, there seems to be three clusters. So let's run the variational algorithm for $K = 3$:

In [16]:
```
# fit the model with dimension D = 2 and K = 3 number of clusters.
vi = VariationalGMM(D=2, K=3, alpha0=10)
vi.fit(Xtrain, max_itt=10000, seed=1)

# compute probabilities p(z_n|x_n) (for the training data, we could also have used vi.r instead of ptrain)
ptrain = vi.compute_component_probs(Xtrain)
ztrain = jnp.argmax(ptrain, axis=1)

# prepare grid for log predictive density (lpd)
P = 100
x_grid = jnp.linspace(-10, 10, P)
XX, YY = jnp.meshgrid(x_grid, x_grid)
Xp = jnp.column_stack((XX.ravel(), YY.ravel()))
lpd = vi.evaulate_log_predictive(Xp, pointwise=True).reshape((P, P))

# evaluate log predictive density for test set
test_lpd = vi.evaulate_log_predictive(Xtest)
print(f'Test LPD for K={vi.K}: {test_lpd:4.3f}')

# prep. plot
fig, axes = plt.subplots(1, 4, figsize=(25, 6))
# plot mixing weights
barlist = axes[0].bar(jnp.arange(1, vi.K+1), vi.pi)

for k in range(vi.K):
    barlist[k].set_color(colors[k])

# plot data, clusters and assignments
plot_data(ax=axes[1], test=False, legend=False)
handles, labels = [], []
for k in range(vi.K):
    plot_std_dev_contour(axes[1], vi.m[k], vi.S[k], facecolor=colors[k], alpha=float(vi.pi[k]/jnp.max(vi.pi)), label=f'k ={k+1}')
    hk = axes[2].plot(Xtrain[ztrain==k, 0], Xtrain[ztrain==k, 1], '.', color=colors[k])
    handles.append(hk[0])
    labels.append('k = %d' % (k+1))

# plot predictive density
axes[3].pcolormesh(x_grid, x_grid, jnp.exp(lpd), cmap=plt.cm.RdBu_r, shading='auto')


# labels, titles etc
axes[0].set(title='Posterior mean of mixture weights', xlabel='Component index $k$');
axes[1].set(title='Estimated components', xlabel='$x_1$', ylabel='$x_2$')
axes[2].set(title='Component assignments', xlim=(-10, 10), ylim=(-10, 10), xlabel='$x_1$', ylabel='$x_2$')
axes[3].set(xlabel='$x_1$', title='Log predictive density')
```

```
fig.legend(handles, labels, loc='center right', markerscale=3);
fig.subplots_adjust(hspace=0.3, wspace=0.3)
```

```
Converged in   36 iterations for K =  3 with lowerbound = -4546.178. Time = 10.37
Test LPD for K=3: -4533.747
```



Task 4.1: Explain what you see in the three panels. What is the value of the lower bound for $K = 3$? [**Discussion question**]

Task 4.2: What happens if you run the analysis with $K = 10$? Explain what you see. What is the value of the lower bound for $K = 10$? [**Discussion question**]

Notes

- the algorithm may take a bit longer to run for $K = 10$
- the plotting code does not work for $K \geq 11$ due to the chosen color scheme

Task 4.3: In the simulation, we have used a Dirichlet prior with hyperparameter value $\alpha_0 = 10$. What happens of you decrease it to $\alpha_0 = 0.1$? What is the value of the lower bound? [**Discussion question**]

Task 4.4: What is the interpretation of the α_0 parameter and how does it affect the prior distribution of the mixing weights π? Use eq. (10.69) in Bishop above to explain your reasoning. [**Discussion question**]

Task 4.5: What happens if you run the analysis multiple times with different seeds? Why does the assigned clusters change from run to run, but not the predictive density? [**Discussion question**]

Task 4.6: Only one of the following to questions is meaningful to answer. Which one and why?

- What is the posterior probability of the 7'th data point belonging to the 2-nd cluster?
- What is the posterior probability of the 7'th and 9'th data points beloging to the same cluster?

-

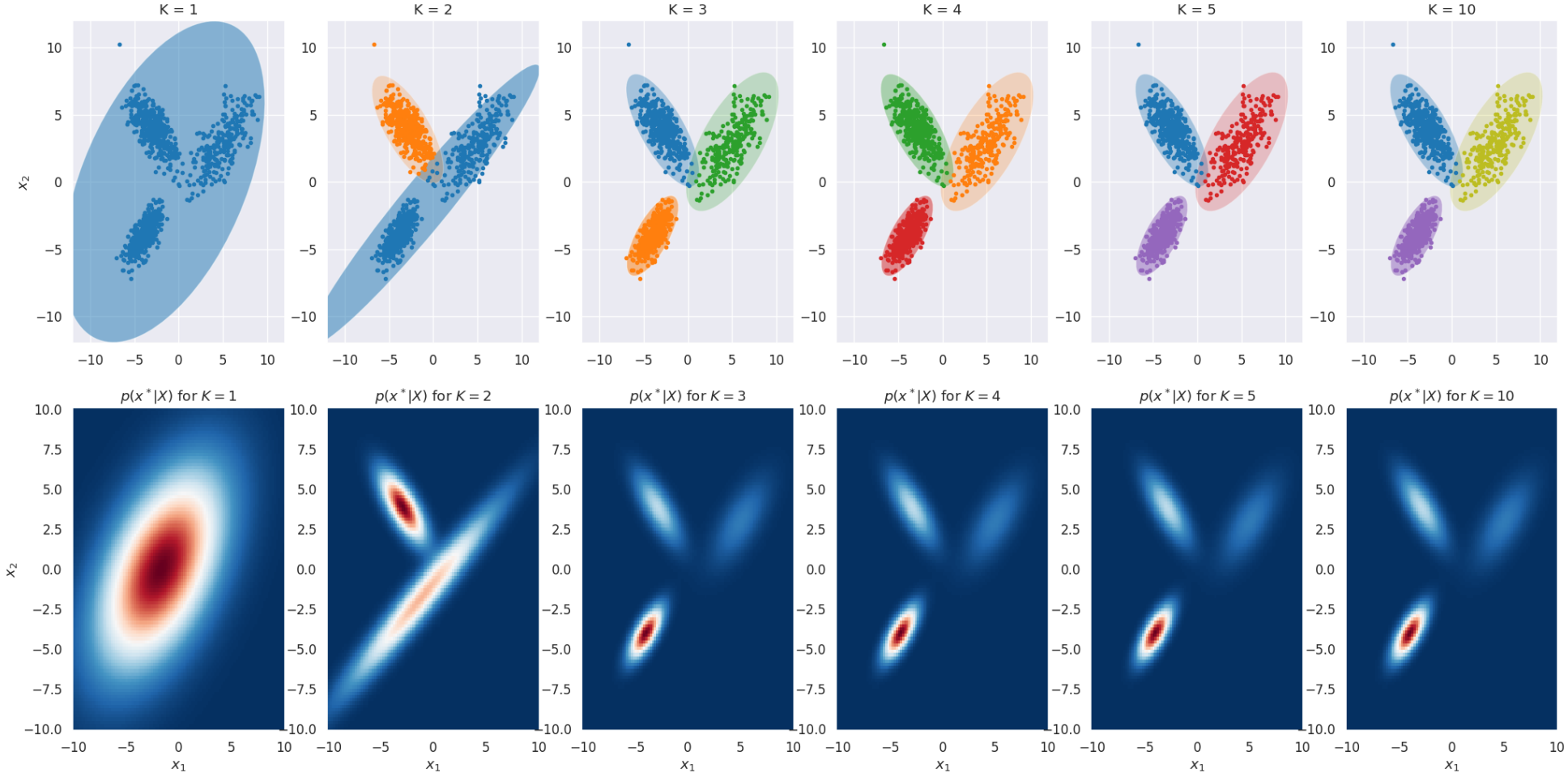
Model selection

We can choose an optimal value for K by measuring the log predictive density $p(x^*|\mathbf{X})$ using eq. (10.80) in Bishop for an independent test/validation set as done below. First, let's repeat and plot the above figure for $K \in [1, 2, 3, 4, 5, 10]$ for $\alpha_0 = 0.1$.

In [17]:
```
# fit models for K = 1, .., 5, 10
Ks = [1, 2, 3, 4, 5, 10]
fits = [VariationalGMM(D=2, K=K, alpha0=0.1).fit(Xtrain, max_itt=10000) for K in Ks]

# prepare grid
P = 100
x_grid = jnp.linspace(-10, 10, P)
XX, YY = jnp.meshgrid(x_grid, x_grid)
Xp = jnp.column_stack((XX.ravel(), YY.ravel()))

# plot
fig, axes = plt.subplots(2, len(fits), figsize=(25, 12))
for idx_fit, fit in enumerate(fits):

    # compute class assignments for test set
```

```
    z = jnp.argmax(fit.compute_component_probs(Xtest), axis=1)

    # compute lpd
    lpd = fit.evaulate_log_predictive(Xp, pointwise=True).reshape((P, P))

    # plot
    for k in range(fit.K):
        plot_std_dev_contour(axes[0, idx_fit], fit.m[k], fit.S[k], facecolor=colors[k], alpha=float(0.5*fit.pi[k]/jnp.max(fit.pi)), label=f'k ={k+1}')
        axes[0, idx_fit].plot(Xtest[z==k, 0], Xtest[z==k, 1], '.', color=colors[k])
        axes[0, idx_fit].set(title='K = %d' % fit.K, ylim=(-12, 12), xlim=(-12, 12))
        axes[1, idx_fit].pcolormesh(x_grid, x_grid, jnp.exp(lpd), cmap=plt.cm.RdBu_r, shading='auto')
        axes[1, idx_fit].set(xlabel='$x_1$', title='$p(x^*|X)$ for $K = %d$' % fit.K)
    axes[0, 0].set_ylabel('$x_2$')
    axes[1, 0].set_ylabel('$x_2$')
```

```
Converged in    1 iterations for K =  1 with lowerbound = -6113.431. Time = 2.38
Converged in   39 iterations for K =  2 with lowerbound = -5112.039. Time = 5.23
Converged in   33 iterations for K =  3 with lowerbound = -4550.666. Time = 3.83
Converged in   90 iterations for K =  4 with lowerbound = -4551.668. Time = 15.78
Converged in   75 iterations for K =  5 with lowerbound = -4552.595. Time = 16.78
Converged in   57 iterations for K = 10 with lowerbound = -4556.680. Time = 22.82
```



Task 4.7: Compute the average log predictive density for the test for each K and plot it as a function of K. What is the optimal number of clusters?

Hint: use the function `evaulate_log_predictive` from the `VariationalGMM` object

Part 4: Clustering images using the Bayesian Gaussian Mixture Model

The goal of this part is to apply the Bayesian mixture model to cluster images. We will work with the same data as in exercise 7. That is, a subset of images from the Linnaeus 5 dataset (http://chaladze.com/l5/), where we have use a ResNet18 network as feature extractor. See Exercise 7 for more details. The main difference is now that we won't feed the image labels to the algorithm, but only the image features.

First, we load the data and reduce to the dimensionality using PCA.

In [18]:
```
data = jnp.load('./ex7_data.npz')
labels = list(data['labels'])
features = data['features']
targets = data['targets']
num_classes = data['num_classes'][()]

X = PCA_dim_reduction(features, num_components=10)
N, D = X.shape

# split into training/test
key = random.PRNGKey(222)
Ntrain = int(0.8*N)
Ntest = N - Ntrain

idx_train = random.choice(key, jnp.arange(N), shape=(Ntrain,), replace=False)
idx_test = jnp.setdiff1d(jnp.arange(N), idx_train)

print('Number of images: %d' % N)
print('Number of features: %d' % D)

Xtrain, Xtest = X[idx_train], X[idx_test]
ttrain, ttest = targets[idx_train], targets[idx_test]
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[18], line 1
----> 1 data = jnp.load('./ex7_data.npz')
      2 labels = list(data['labels'])
      3 features = data['features']

File ~/miniconda3/envs/02477_Bayesian_Machine_Learning/lib/python3.13/site-packages/jax/_src/numpy/lax_numpy.py:246, in load(file, *args, **kwargs)
    211 """Load JAX arrays from npy files.
    212
    213 JAX wrapper of :func:`numpy.load`.
   (...)    242   Array([2, 4, 6, 8], dtype=bfloat16)
    243 """
    244 # The main purpose of this wrapper is to recover bfloat16 data types.
    245 # Note: this will onlv work for files created via np.save(), not np.savez().
--> 246 out = np.load(file, *args, **kwargs)
    247 if isinstance(out, np.ndarray):
    248   # numpy does not recognize bfloat16, so arrays are serialized as void16
    249   if out.dtype == 'V2':

File ~/miniconda3/envs/02477_Bayesian_Machine_Learning/lib/python3.13/site-packages/numpy/lib/_npyio_impl.py:454, in load(file, mmap_mode, allow_pickle, fix_imports, encoding, max_header_size)
    452     own_fid = False
    453 else:
--> 454     fid = stack.enter_context(open(os.fspath(file), "rb"))
    455     own_fid = True
    457 # Code to distinguish from NumPy binary files and pickles.

FileNotFoundError: [Errno 2] No such file or directory: './ex7_data.npz'
```

and let's visualize some examples.

In [ ]:
```
def show_example(ax, i, title=True):
    img = Image.open('./images/%d.jpg' % i)
    target = targets[i]
    label = labels[int(target)]
    ax.imshow(img)
    if title:
        ax.set_title(label)
    ax.grid(False)
    ax.axis('off')

fig, ax_ = plt.subplots(2, 5, figsize=(20, 6))
ax = ax_.flat
for i in range(10):
    show_example(ax[i], i)
```

Recall that the images come from a dataset containing the following four labels: berry, flower, dog, bird. Let's fit a Bayesian GMM with $K = 4$ to the training set and inspect images from the identified components.

In [ ]:
```
vi = VariationalGMM(D=D, K=4, alpha0=1e-3).fit(Xtrain.squeeze(), max_itt=10000, verbose=True)
```

Let's plot the posterior mean of the mixing weights

In [ ]:
```
fig, ax = plt.subplots(1, 1, figsize=(8, 2))
ax.bar(jnp.arange(1, vi.K+1), vi.pi)
ax.set(xlabel='Component index', ylabel='Posterior mean for $\\pi_k$', title='Posterior mean of mixing weights', xticks=jnp.linspace(1, vi.K, min(vi.K, 20)).round());
```

The code below visualizes images from some of the larger clusters (containing 30 images or more)

Task 4.8: Does the four clusters align with the target labels of the data set? [**Discussion question**]

Task 4.9: Increase the number of clusters to $K = 50$ and repeat the analysis. Can you spot any patterns representative for each cluster? [**Discussion question**]

- Note: We allow for a maximum of 10000 iterations, but it should converge much earlier.

In [ ]:
```
def visualize_img_clustering(z, K, idx_list):

    for k in range(K):

        # find and count all images assigned to cluster k
        idx_k = jnp.where(z == k)[0]

        num_images_in_cluster = len(idx_k)

        if num_images_in_cluster < 30:
            continue

        # visualiz
        fig, axes = plt.subplots(3, 5, figsize=(20, 12.5))

        for idx_plot, idx in enumerate(idx_k):
            if idx_plot >= jnp.prod(jnp.array(axes.shape)):
                break

            show_example(axes.flat[idx_plot], idx_list[idx], False)

        fig.subplots_adjust(hspace=0.0, wspace=0.00)
        fig.suptitle('Cluster idx %d with %d images' % (k+1, num_images_in_cluster), fontweight='bold')


ptrain = vi.compute_component_probs(Xtrain)
ztrain = jnp.argmax(ptrain, axis=1)
visualize_img_clustering(ztrain, vi.K, idx_train)
```

In [ ]:

In [ ]: